

CSEN 79: Make Decision Trees with Gini Index

Background

Decision tree is one of the “supervised learning” AI techniques. The idea is to generate, or refine, a decision tree with sets of training data. Then the software can categorize new records correctly as if it is intelligent.

Gini Index of Impurity

The Gini Index of Impurity, commonly known as Gini Impurity, quantifies the homogeneity of a data set. Although it shares mathematical foundations with the well-known Gini Coefficient—which measures economic inequality within a society—the two serve distinct purposes. A data set is considered “pure” when a majority of its records belong to the same classification. Conversely, the concept of “entropy” reflects the diversity of classifications present; higher entropy signals lower purity.

Mathematically, the Gini Index of impurity is defined as $G = 1 - \sum p_i^2$, where p_i represents the probability that a record falls into a particular classification. When p_i is 100%, indicating all records share the same classification, the Gini Index equals zero. When dividing the data set into multiple subsets, the overall Gini Index is calculated as the weighted average of the indices of each subset: $G = \sum w_j G_j$, where w denotes the probability that an element belongs to a specific subset.

A reduction in the combined Gini index after division indicates “information gain,” signifying that segmentation has provided additional insight into the data set’s structure. For example, if grains of sand appear grey in aggregate but are separated into distinct piles of black and white, the process yields valuable information about the composition of the original mixture.

Decision Tree

We use decision tree to make decisions every day in the form of “if this happens, I will make that decision, otherwise a different one.” For example, “if the grill line is too long at the cafeteria, I will do salad bar.”

Put differently, a decision tree is a categorization system based on a set of conditions. The possible decisions reduce as one descend the tree. The final categorizations are the leaf nodes.

We can use Gini Impurity to generate decision trees.

Given a data set in which each record has a set of attributes. We may be able to categorize these records based on the attributes. For the above example, the “record” is each day’s lunch choice. The attribute for the record the length of queue at the grill line. And the categorization is “grill” or “salad bar.” After a few weeks of recording your lunch decisions, we compute the Gini Index for those data and may predict “if the line is longer than 5, you are likely to eat salad.”

(There are other algorithms for splitting the records other than Gini Index. Modern AI engines typically use ID3 (*Iterative Dichotomiser 3*) or its successors: C4.5 or C5.0.)

CSEN 79: Make Decision Trees with Gini Index

We use decision trees in our daily decision-making by following a structure of conditional choices—essentially, “if this happens, I will do that; otherwise, I’ll do something else.” For example, “If the grill line is too long at the cafeteria, I will choose the salad bar.”

In more precise terms, a decision tree is a system for categorizing data based on a series of conditions applied to its attributes. As you move down the tree, each decision point (or node) further narrows the possible outcomes, with the final categorizations represented by the leaf nodes.

Gini Impurity is a common criterion used to generate decision trees. Given a data set in which each record consists of several attributes, we may be able to categorize these records based on those attributes. In the example above, each “record” represents a day’s lunch choice. The attribute for the record is the length of the queue at the grill line, and the categorization (label) is either “grill” or “salad bar.” After collecting several weeks of lunch decisions, we compute the Gini Impurity for those data and may predict, for instance, “if the line is longer than 5, you are likely to eat salad.”

While Gini Impurity is commonly used for splitting nodes in decision trees, other algorithms use different criteria. For example, the ID3 algorithm relies on information gain, while C4.5 and C5.0 are later algorithms that improve upon ID3 by better handling continuous attributes and missing values. These alternatives provide additional flexibility and accuracy in various data scenarios.

Assignment

Given a set of training data, generate the decision tree to be used for the categorization of future similar records.

Each record of the training data has 3 “attributes”, which are integers in the range of 1,100, and the “category” which is either “red” or “blue.” The decision tree takes the form of a binary tree: each node has two sub-nodes. The “root” of the decision tree has all the records. It’s two sub-nodes each has a subset. The tree descends similarly afterward.

You are to find the decision tree with the least combined Gini Index of Impurity.

Specifically:

Download the provided code as in “decisiontree.tar.” Unpack and build to make sure you get the whole thing. Note the package includes several test data files. Implement those functions left blank in treedata.cxx and treeio.cxx. Test with provided data files.

1. The formatted output operator (<<) for “TreeData” should print out the final decision tree in a human understandable form. For example, it could be a series of sentences like:

“If attribute 0 is greater than 67 and attribute 1 is less than 30 and attribute 2 is greater than 73, the category is Red. Otherwise, if”

CSEN 79: Make Decision Trees with Gini Index

2. The formatted input operator (`>>`) fills in the “records” member for `TreeData`. Note this member is typed as `std::shared_ptr`. You may change the type as you see fit. If you do so, also modify the destructor if necessary.
3. Function `TreeData::split` returns a pair of `TreeData` which “split” “this” into two (without changing it.)
4. Function `TreeData::huntSplit` finds the optimal split point for the optimal attribute. It returns `HuntType` which is a `std::pair`.
5. Function `TreeData::makeTree()` transforms the root node into a decision tree.
6. If you wish to change the class structure, like adding more members, go ahead and highlight those with comments.
7. There should be no reason to change “main” and “record”. But if you wish to, go ahead.

As always, submit source code. No need to submit test data.

Hints

1. The provided code use STL container “pair.” You should understand it first.
2. The stream input operator “`>>`” populate the root node with a list of record. The “`<<`” operator does the formatted output for the node and should recurse for the entire tree.
3. Each node has a list of records as its data member. It has two descendants that split the records of their parent node. Since there are large number of records, the provided code use pointers to conserve memory usage, in fact `std::shared_ptr`.
4. How to choose the “split point” of one of the attributes? You must attempt to find the optimal split for each attribute, compute the combined Gini Index after the split, and see which is the best.

Alternatively, you can choose an arbitrary attribute to split, then another one for the next level. Then repeat for different sequence until you find the best one. Since this is a “trial-and-error” approach, genetic programming technique may help.

5. To find the optimal split point, “plot” the combined Gini index for each possible split point and look for the “valley” of that curve. The valley is when the delta of two consecutive split points go down then up. (Mathematically, find the point that the 2nd derivative is zero.)

Foods for Thought

In practice, training datasets often contain millions of records with many attributes and more than two categories. These datasets may be imperfect or biased, and memory and computational resources are crucial concerns. Even with flawless data, decision trees can fail if relevant attributes are missing; this exemplifies the “garbage in, garbage out” principle. Furthermore, biased training data—for example, based only on one demographic—may not generalize to other groups.